

Graduate Algorithms

Tutorial Sheet 1: Solutions

August 24, 2026

Note

- Other correct solutions exist.

Problem 1: Greatest Prime Divisor Array

Suggested/Proposed by Vardhan Kumar Ray

For a positive integer $i \geq 2$, let $\text{gpd}(i)$ denote the greatest prime divisor of i . Define an array $P[0 \dots n]$ by

$$P[0] = 0, \quad P[1] = 1, \quad P[i] = \text{gpd}(i) \text{ for } 2 \leq i \leq n.$$

(The values at 0 and 1 are fixed by convention, since neither has a prime divisor.) For example, $n = 10$ gives

$$P = [0, 1, 2, 3, 2, 5, 3, 7, 2, 3, 5].$$

We work in the following cost model. Assigning a value to a variable, adding two integers, comparing two integers, and reading or writing a single array entry given its index each cost one step. **Division and modulo are *not* assumed to cost a constant number of steps**, so you may not use them freely.

- Design a procedure $\text{GREATESTDIVISORS}(n)$ that takes a positive integer n and returns the array P , using at most $cn \log n$ steps for some constant c independent of n . Write it as pseudocode.
- Prove that your procedure returns the array P as defined above.
- Prove that it runs within the stated bound in this cost model.

Solution.

(a) The algorithm

Algorithm 1 Greatest prime divisor array

```
1: function GREATESTDIVISORS( $n$ )
2:   for  $i = 0$  to  $n$  do
3:      $P[i] \leftarrow 0$ 
4:   end for
5:    $P[1] \leftarrow 1$ 
6:   for  $p = 2$  to  $n$  do
7:     if  $P[p] = 0$  then  $\triangleright p$  is prime
8:        $j \leftarrow p$ 
9:       while  $j \leq n$  do
10:         $P[j] \leftarrow p$ 
11:         $j \leftarrow j + p$ 
12:      end while
13:    end if
14:  end for
15:  return  $P$ 
16: end function
```

We cannot ask "does p divide i ?" directly, because that needs a division. But we can visit all multiples of p using addition alone. If we sweep p upwards through the primes and, for each p , write p into every multiple of p , then each entry $P[i]$ gets overwritten once per prime divisor of i , in increasing order. Whatever is left at the end is the *largest* one.

Finally we need to know which p are prime. At the moment we reach p , the entry $P[p]$ is still 0 exactly when no smaller prime has written into it.

(b) Correctness

(1) Every inner loop starts at $j = p \geq 2$ and increases j , so indices 0 and 1 are never written after initialisation. Thus $P[0] = 0$ from line 3, and $P[1] = 1$ from line 5, as required.

(2) We prove by strong induction on p that when the loop reaches p , we have $P[p] = 0$ if and only if p is prime.

Suppose the statement holds for all values below p .

If p is composite, let q be its smallest prime divisor, so $2 \leq q < p$ and $q \mid p$. By the induction hypothesis, iteration q recognized q as prime and ran its inner loop, which visits $q, 2q, 3q, \dots$ up to n . Since $q \mid p$ and $p \leq n$, the value p is one of those multiples, so that loop executed $P[p] \leftarrow q \neq 0$. Nothing ever writes 0 into an entry after the initialisation, so $P[p] \neq 0$ when we reach p .

If p is prime, the only writes so far were made by inner loops belonging to primes $q < p$, and such a loop writes only into multiples of q . As p is prime and $q < p$, we have $q \nmid p$, so no such write touched $P[p]$. Also line 5 wrote only to index 1. Hence $P[p]$ is still its initial value 0.

(3) Fix i with $2 \leq i \leq n$. By Part (2), the iterations that run an inner loop are exactly those with p prime. So the sequence of values written into $P[i]$ over the whole run is

$$p_1 < p_2 < \dots < p_r,$$

the prime divisors of i listed in increasing order, because the outer loop visits p in increasing order. Since $i \geq 2$, it has at least one prime divisor, so $r \geq 1$ and $P[i]$ is written at least once. Every prime divisor of i is at most $i \leq n$, so all of them occur inside the loop range and none is missed. The value remaining at the end is the last one written, namely $p_r = \text{gpd}(i)$. ■

(c) Running time

- Lines 2–4: the loop runs $n + 1$ times and does one array write per iteration, so $O(n)$ steps in total.
- Line 5: one step.
- Lines 6–13: the outer loop body runs $n - 1$ times. The test on line 6 is one array read and one comparison, so the tests alone cost $O(n)$.
- The inner loop of a prime p runs once for each multiple of p that is at most n , that is, $\lfloor n/p \rfloor$ times, and each iteration costs a constant number of steps.

So the total cost of all inner loops is proportional to

$$\sum_{\substack{p \leq n \\ p \text{ prime}}} \left\lfloor \frac{n}{p} \right\rfloor \leq \sum_{k=2}^n \frac{n}{k} = n(H_n - 1) \leq n \ln n,$$

where $H_n = \sum_{k=1}^n 1/k \leq 1 + \ln n$ is the harmonic sum. Adding the $O(n)$ from the other lines, the whole procedure runs in at most $cn \log n$ steps for a suitable constant c , as required. ■

Remark. The bound above is generous. Restricting the sum to primes and using $\sum_{p \leq n} 1/p = \ln \ln n + O(1)$ gives the sharper $\Theta(n \log \log n)$.

Problem 2: Recursion Trees

Suggested/Proposed by *Vardhan Kumar Ray*

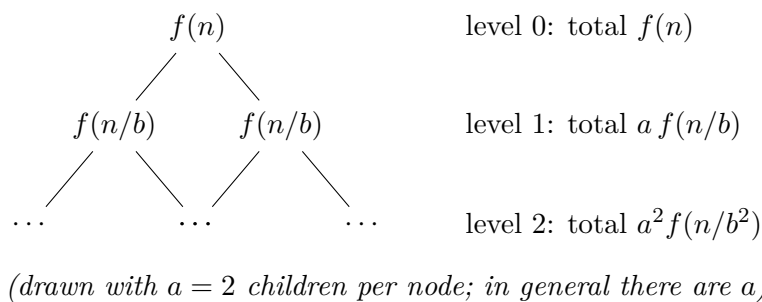
Solve each of the following recurrences using a recursion tree, giving a Θ bound. In each case say which level of the tree carries the dominant cost and why. You may assume $T(a) = \Theta(1)$ for a below some constant, and you may ignore floors and ceilings.

(a) $A(n) = 5A(n/5) + n^2$

(b) $B(n) = B(n/4) + B(n/3) + B(n/6) + n$

Background: how a recursion tree works

Each node stands for one subproblem, labelled with the cost that subproblem pays *by itself*, excluding its recursive calls. The root is the original problem of size n and carries the cost $f(n)$; its children are the subproblems it calls, and so on down to leaves of constant size. The total cost is the sum of all the labels, and the useful way to add them up is level by level.



Write c_t for the total cost of level t and let d be the depth of the tree, so that the answer is $\sum_{t=0}^d c_t$. Three patterns cover most recurrences you will meet.

- **Root-dominated:** The level costs shrink geometrically, say $c_{t+1} \leq r c_t$ for a fixed $r < 1$. Then the sum is within a constant factor of its first term, and $T(n) = \Theta(c_0) = \Theta(f(n))$.
- **Balanced:** Every level costs about the same, $c_t = \Theta(f(n))$. Then the sum is the cost of one level times the number of levels, $T(n) = \Theta(f(n) d)$, and $d = \Theta(\log n)$ whenever the subproblem size shrinks by a constant factor each step.
- **Leaf-dominated:** The level costs grow geometrically. Then the sum is within a constant factor of its *last* term, so $T(n) = \Theta(c_d)$. The bottom level consists of the leaves, each costing $\Theta(1)$, so this is $\Theta(\# \text{leaves})$; for $T(n) = aT(n/b) + f(n)$ that is $\Theta(a^{\log_b n}) = \Theta(n^{\log_b a})$.

A recursion tree gives the guess quickly. Where the tree is unbalanced, it is easier to confirm the guess by substitution, which is what we do in part (b).

Solution.

(a) $A(n) = 5A(n/5) + n^2$

At level k there are 5^k subproblems, each of size $n/5^k$, and each pays $(n/5^k)^2$. So the cost of level k is

$$5^k \cdot \left(\frac{n}{5^k}\right)^2 = \frac{n^2}{5^k}.$$

level k	subproblems	size of each	cost of the level
0	1	n	n^2
1	5	$n/5$	$n^2/5$
2	25	$n/25$	$n^2/25$
k	5^k	$n/5^k$	$n^2/5^k$

The level costs form a geometric series with ratio $1/5 < 1$, so summing over all $\log_5 n$ levels,

$$\sum_{k \geq 0} \frac{n^2}{5^k} \leq n^2 \cdot \frac{1}{1 - 1/5} = \frac{5}{4} n^2.$$

The bottom level has $5^{\log_5 n} = n$ leaves of constant cost, contributing only $\Theta(n)$. Since the root alone already costs n^2 , we get

$$A(n) = \Theta(n^2),$$

and the **root** carries the dominant cost.

(b) $B(n) = B(n/4) + B(n/3) + B(n/6) + n$

Here the three subproblems have *different* sizes, so the tree is unbalanced and different branches bottom out at different depths.

A subproblem of size s pays/spends s and passes sizes $s/4$, $s/3$, $s/6$ to its children, which total

$$\frac{s}{4} + \frac{s}{3} + \frac{s}{6} = \frac{3s}{4}.$$

So the total size at each level is $3/4$ of the level above, and since each node pays exactly its own size, the cost of level k is $(3/4)^k n$.

level k	subproblems	cost of the level
0	1	n
1	3	$\frac{3}{4}n$
2	9	$\frac{9}{16}n$
k	3^k	$\left(\frac{3}{4}\right)^k n$

Summing the geometric series gives $\sum_{k \geq 0} (3/4)^k n = 4n$, suggesting $B(n) = \Theta(n)$.

Because the tree is unbalanced, let us confirm the upper bound by substitution. Claim: $B(m) \leq 4m$ for all m large enough. Assuming it for all sizes below n ,

$$B(n) \leq 4 \cdot \frac{n}{4} + 4 \cdot \frac{n}{3} + 4 \cdot \frac{n}{6} + n = 4 \cdot \frac{3n}{4} + n = 3n + n = 4n,$$

so the claim holds at n too, and the constant 4 never grows. The base cases are covered by choosing the constant large enough. For the lower bound, the root alone costs n . Hence

$$B(n) = \Theta(n),$$

again **root**-dominated.

Remark: The reason is that $\frac{1}{4} + \frac{1}{3} + \frac{1}{6} = \frac{3}{4} < 1$. Had the fractions summed to exactly 1, as in $B(n/2) + B(n/3) + B(n/6) + n$, every level would cost n , there would be $\Theta(\log n)$ levels, and the answer would be $\Theta(n \log n)$ instead.

Problem 3: Stooge Sort

Suggested/Proposed by Vardhan Kumar Ray

Stooge sort is a sorting algorithm notable for being remarkably slow. To sort a segment $A[i \dots j]$ it first compares the two end entries and swaps them if they are out of order; then, provided the segment has more than two entries, it recursively sorts three overlapping segments, each of about two-thirds the length.

Algorithm 2 Stooge sort, called as $\text{STOOGESORT}(A, 1, n)$

```

1: function STOOGESORT( $A, i, j$ )
2:   if  $A[i] > A[j]$  then
3:     swap  $A[i]$  and  $A[j]$ 
4:   end if
5:   if  $j - i + 1 > 2$  then
6:      $k \leftarrow \lfloor (j - i + 1)/3 \rfloor$ 
7:     STOOGESORT( $A, i, j - k$ )
8:     STOOGESORT( $A, i + k, j$ )
9:     STOOGESORT( $A, i, j - k$ )
10:  end if
11: end function

```

- Write down a recurrence for the running time of STOOGESORT on an array of n entries, and solve it to show that the running time is $O(n^{\log_{1.5} 3})$. Say which part of the recursion tree carries the dominant cost.
- Give an upper bound on the total number of swaps the algorithm performs.

Solution.

(a) The recurrence and its solution

Let $n = j - i + 1$ be the length of the segment and let $T(n)$ be the running time. Outside the recursive calls the procedure does a constant amount of work.

With $k = \lfloor n/3 \rfloor$, each of the three recursive calls is on a segment of length $n - k = n - \lfloor n/3 \rfloor$, which is about $2n/3$. Ignoring floors,

$$T(n) = 3T\left(\frac{2n}{3}\right) + \Theta(1), \quad T(n) = \Theta(1) \text{ for } n \leq 2.$$

Every node has three children, and the segment length shrinks by a factor $2/3$ at each step, so at level t there are 3^t nodes, each of length $(2/3)^t n$ and each costing $\Theta(1)$. The tree bottoms out when $(2/3)^t n$ is constant, that is at depth

$$d = \log_{3/2} n.$$

level t	nodes	length of each	cost of the level
0	1	n	$\Theta(1)$
1	3	$2n/3$	$\Theta(3)$
2	9	$4n/9$	$\Theta(9)$
t	3^t	$(2/3)^t n$	$\Theta(3^t)$

Summing the levels gives a geometric series with ratio $3 > 1$, so the total is dominated by its last term:

$$T(n) = \Theta\left(\sum_{t=0}^d 3^t\right) = \Theta(3^d) = \Theta(3^{\log_{3/2} n}) = \Theta(n^{\log_{3/2} 3}),$$

the last step using the identity $a^{\log_b c} = c^{\log_b a}$. Since $3/2 = 1.5$, this is $\Theta(n^{\log_{1.5} 3})$.

Check with the Master theorem (See Quiz 2 Problem 1 for the statement): Here $a = 3$, $b = 3/2$ and $f(n) = \Theta(1)$, so the critical exponent is $n^{\log_{3/2} 3}$ with $\log_{3/2} 3 \approx 2.71$. Taking any $\varepsilon < 2.71$ we have $f(n) = \Theta(1) = O(n^{\log_{3/2} 3 - \varepsilon})$, so Case 1 applies and $T(n) = \Theta(n^{\log_{3/2} 3})$, agreeing with the tree.

(b) The number of swaps

A swap happens only at line 3, and only when $i < j$ and $A[i] > A[j]$. Recall that a pair of positions $p < q$ is an inversion of the array if $A[p] > A[q]$, and write $\text{inv}(A)$ for the number of inversions.

Claim. Exchanging the entries at an inverted pair of positions strictly decreases $\text{inv}(A)$.

Proof. Let $p < q$ with $a = A[p] > A[q] = b$, and exchange them. Sort the pairs of positions into four kinds.

- *The pair (p, q) itself.* It was an inversion and is not one afterwards, since $b < a$. This contributes -1 .
- *Pairs avoiding both p and q .* Neither entry moved, so nothing changes.
- *Pairs joining an outside position m to p or to q ,* where $m < p$ or $m > q$. Such an m lies on the same side of both positions, and the two values a and b have merely traded places, so the number of inversions among these two pairs depends only on the set $\{a, b\}$ and is unchanged.
- *Pairs joining a position m strictly between them,* so $p < m < q$; write $c = A[m]$. Before the swap the relevant pairs are (p, m) and (m, q) , contributing $[a > c] + [c > b]$; afterwards they contribute $[b > c] + [c > a]$. If $b < c < a$, the count drops from 2 to 0. In every other case ($c < b$ or $c > a$) the count is 1 before and 1 after, or 0 and 0.

Adding up, the swap changes the number of inversions by exactly

$$-1 - 2 \cdot \left| \{ m : p < m < q \text{ and } b < A[m] < a \} \right| \leq -1. \quad \square$$

The number of inversions is a non-negative integer; before the algorithm starts it is at most the total number of pairs,

$$\text{inv}(A) \leq \binom{n}{2} = \frac{n(n-1)}{2},$$

attained when the array is in reverse order. Every swap strictly decreases it, and it can never go below 0. Hence the algorithm performs at most

$$\binom{n}{2} = O(n^2)$$

swaps in total.

Remark. This is much smaller than the running time $\Theta(n^{\log_{1.5} 3}) \approx \Theta(n^{2.71})$, and it makes one comparison per invocation, so it performs $\Theta(n^{2.71})$ comparisons, but at most $O(n^2)$ of them can ever result in a swap. Almost all of its work is looking at pairs that are already in the right order.

A weaker bound follows: each invocation performs at most one swap, so the number of swaps is at most the number of invocations, which satisfies $C(n) = 3C(2n/3) + 1$ and hence is $\Theta(n^{\log_{1.5} 3})$.

Had we charged $\Theta(n)$ per invocation instead of $\Theta(1)$, the recurrence would be $T(n) = 3T(2n/3) + \Theta(n)$, and since $n = O(n^{2.71-\varepsilon})$ the Master theorem would still give Case 1 and the same answer $\Theta(n^{\log_{1.5} 3})$.

Problem 4: Finding a Fixed Point

Suggested/Proposed by Vardhan Kumar Ray

Let $X[1 \dots n]$ be an array of n distinct integers, possibly negative, sorted in strictly increasing order. Call an index k a *fixed point* if $X[k] = k$.

- (a) Give an algorithm that either returns a fixed point of X or correctly reports that none exists, and that is asymptotically faster than examining every entry. State and justify its running time.
- (b) Now suppose you are told in advance that $X[1] > 0$. Give a *substantially* faster algorithm for the same task, and justify both its correctness and its running time.

Background: binary search

Binary search finds a target in a sorted array by repeatedly halving the range that could still contain it. It keeps two indices lo and hi with the invariant *if the target is anywhere, it is in $X[lo \dots hi]$* . Probing the midpoint m either succeeds, or tells us the target lies strictly to one side, and that side is discarded. The range shrinks from n to $n/2$ to $n/4$ and so on, so after at most $\lfloor \log n \rfloor + 1$ probes it is empty and we can report failure. The requirement is only that the array is sorted.

Solution.

(a) An $O(\log n)$ algorithm

Define

$$D[i] = X[i] - i, \quad 1 \leq i \leq n.$$

A fixed point is exactly an index with $D[i] = 0$, so we want to search D for the value 0. We never build D ; we compute an entry when we need it, at the cost of one array read and one subtraction.

D is sorted (non-decreasing). The entries of X are integers in strictly increasing order, so $X[i+1] \geq X[i] + 1$. Therefore

$$D[i+1] = X[i+1] - (i+1) \geq (X[i] + 1) - i - 1 = X[i] - i = D[i].$$

So we may binary search D for 0:

Algorithm 3 Fixed point by binary search

```
1:  $lo \leftarrow 1, \quad hi \leftarrow n$ 
2: while  $lo \leq hi$  do
3:    $m \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
4:   if  $X[m] - m = 0$  then
5:     return  $m$ 
6:   else if  $X[m] - m < 0$  then
7:      $lo \leftarrow m + 1$ 
8:   else
9:      $hi \leftarrow m - 1$ 
10:  end if
11: end while
12: return "no fixed point"
```

Correctness. We maintain the invariant that every fixed point of X lies in $X[lo \dots hi]$. It holds initially. If $X[m] < m$, then $D[m] < 0$, and since D is non-decreasing, $D[i] \leq D[m] < 0$ for every $i \leq m$; so no index $i \leq m$ is a fixed point and discarding them is safe. The case $X[m] > m$ is

symmetric. When the loop ends the range is empty, so by the invariant there is no fixed point at all.

Running time. Each iteration does $O(1)$ work and halves the length of $[lo, hi]$, so there are $O(\log n)$ iterations: $O(\log n)$ time.

(b) An $O(1)$ algorithm when $X[1] > 0$

Algorithm. Read $X[1]$. If $X[1] = 1$, return the index 1. Otherwise report that no fixed point exists.

Correctness. The entries are integers and $X[1] > 0$, so $X[1] \geq 1$.

If $X[1] = 1$ then index 1 is a fixed point, and the answer is correct.

If $X[1] \geq 2$, then because X is strictly increasing on integers we have $X[i] \geq X[1] + (i - 1)$ for every i , hence

$$X[i] \geq 2 + (i - 1) = i + 1 > i,$$

so no index is a fixed point and reporting failure is correct.

Running time. One array read and one comparison: $O(1)$, independent of n . ■

Problem 5: Finding the Maximum and the Minimum Together

Suggested/Proposed by Vardhan Kumar Ray

We work in the comparison model: an algorithm may compare two elements and learn which is larger, at unit cost. Given $n \geq 2$ distinct elements, the task is to report *both* the maximum and the minimum.

Prove that every deterministic algorithm for this task makes at least $\lceil 3n/2 \rceil - 2$ comparisons in the worst case. Then show the bound is tight by exhibiting an algorithm that always achieves it.

Solution.

The lower bound

An element x can be ruled out as the maximum precisely when some element is known to be above it, and a chain of known relations ending above x must begin with a comparison that x itself lost. So:

$$\begin{aligned} x \text{ is ruled out as the maximum} &\iff x \text{ lost at least one comparison,} \\ x \text{ is ruled out as the minimum} &\iff x \text{ won at least one comparison.} \end{aligned}$$

Give each element two labels, “has won” and “has lost”, initially both absent, and let

$$\Phi = \#\{\text{elements carrying } has\ won\} + \#\{\text{elements carrying } has\ lost\}.$$

Then $\Phi = 0$ at the start and $\Phi = 2(n - 1) = 2n - 2$ at the end. Each comparison can add at most 2 to Φ : the winner may acquire *has won* and the loser may acquire *has lost*, and nothing else changes.

Call an element *untouched* if it carries neither label.

The adversary. We cannot choose the algorithm, but we may choose the input as the algorithm runs. When x and y are compared, we answer by the first rule that applies:

- (1) if one of them has won before but never lost, we declare that one the winner;
- (2) if one of them has lost before but never won, we declare that one the loser;
- (3) otherwise we give any answer consistent with the earlier ones.

Every answer is consistent with some genuine input. Under rule (1) the declared winner has never lost, so no earlier answer places anything above it and declaring it the winner creates no contradiction; and if the pair was already comparable, this was the forced answer anyway. Rule (2) is symmetric. Under rule (3), we take the forced answer if the pair is comparable, and otherwise either answer extends the order.

Suppose x is touched. If rule (1) or (2) applies, the element it names already carries the label its role would give it, so only the other element can gain one. If neither applies, then neither x nor y carries exactly one of the two labels, so x , being touched, carries both and can gain nothing. Either way Φ grows by at most 1.

Only a comparison between two untouched elements can add 2, and such a comparison turns both of them into touched elements. Since there are n elements to begin with, at most $\lfloor n/2 \rfloor$ comparisons of this kind can ever occur. If the algorithm makes t comparisons in total, then

$$2n - 2 = \Phi_{\text{final}} \leq 2 \left\lfloor \frac{n}{2} \right\rfloor + 1 \cdot \left(t - \left\lfloor \frac{n}{2} \right\rfloor \right) = \left\lfloor \frac{n}{2} \right\rfloor + t,$$

so

$$t \geq 2n - 2 - \left\lfloor \frac{n}{2} \right\rfloor = \left\lceil \frac{3n}{2} \right\rceil - 2,$$

the last equality being a check of the two parities: for n even both sides are $\frac{3n}{2} - 2$, and for n odd both are $\frac{3n-3}{2}$. ■

The bound is attained

Process the elements in pairs, keeping a running maximum and minimum.

- If n is even, compare the first two elements once and initialise the running maximum and minimum from them. This costs 1 comparison and consumes 2 elements.
- If n is odd, initialise both the running maximum and minimum to the first element, at no cost, consuming 1 element.

An even number of elements now remains. Take them two at a time: compare the two against each other (1 comparison), then compare the larger with the running maximum and the smaller with the running minimum (2 more). That is 3 comparisons per pair, and crucially the smaller of the two is never compared with the running maximum.

For n even this gives

$$1 + 3 \cdot \frac{n-2}{2} = \frac{3n}{2} - 2,$$

and for n odd it gives

$$0 + 3 \cdot \frac{n-1}{2} = \frac{3n-3}{2}.$$

Both equal $\lceil 3n/2 \rceil - 2$, so the lower bound is tight. ■

Problem 6: A Lower Bound for Convex Hulls

Suggested/Proposed by Vardhan Kumar Ray

Let $S = \{p_1, \dots, p_n\}$ be a set of n points in $\mathbb{R}^d$ with $d \geq 2$. The *convex hull* of S , written $\text{conv}(S)$, is the smallest convex set containing S ; equivalently, it is the set of all convex combinations $\sum_i \lambda_i p_i$ with $\lambda_i \geq 0$ and $\sum_i \lambda_i = 1$. A point of S is a *vertex* of the hull if it is not a convex combination of the other points of S .

The Convex Hull problem takes S as input and must output the vertices of $\text{conv}(S)$ in *boundary order*: for $d = 2$, the vertices listed in counterclockwise order around the hull.

We work in a model where an algorithm may perform arithmetic operations and comparisons at unit cost, and where the known $\Omega(n \log n)$ lower bound for sorting n numbers applies.

Prove that every algorithm for the Convex Hull problem makes $\Omega(n \log n)$ operations in the worst case, already for $d = 2$.

Solution.

We argue by reduction: we show that a fast convex hull algorithm would give a fast sorting algorithm. Since sorting needs $\Omega(n \log n)$ operations in this model, so does convex hull.

The reduction

Let $x_1, \dots, x_n$ be the numbers to be sorted; we may assume they are distinct. Map each one to a point on the parabola $y = x^2$:

$$p_i = (x_i, x_i^2), \quad i = 1, \dots, n,$$

and let $S = \{p_1, \dots, p_n\}$. Run the convex hull algorithm on S , obtaining the hull vertices in counterclockwise order. Scan that list for the point of smallest x -coordinate, and read the list off starting there, cyclically, outputting the x -coordinates. We claim this outputs $x_1, \dots, x_n$ in increasing order.

Correctness

(1) Every input point is a hull vertex: Fix i and consider the tangent line to the parabola at p_i , namely $y = 2x_i x - x_i^2$. For any real x ,

$$x^2 - (2x_i x - x_i^2) = (x - x_i)^2,$$

which is 0 at $x = x_i$ and strictly positive otherwise. So p_i is the unique point of S minimising the linear function $\ell(x, y) = y - 2x_i x$, every other point of S lying strictly above the tangent line. A point that is the unique minimiser of a linear function over a finite set cannot be written as a convex combination of the others, since such a combination would have ℓ -value at least as large as the minimum over the others, which is strictly larger. Hence p_i is a vertex of $\text{conv}(S)$, and this holds for every i : all n points are vertices.

(2) Boundary order is sorted order: Because all the points lie on an upward-opening parabola, the hull is a convex polygon whose lower boundary is the polygonal path joining the points in order of increasing x -coordinate, while its upper boundary is the single edge joining the leftmost point to the rightmost one. Traversing the boundary counterclockwise runs along the lower boundary from left to right. So, starting the list at the point of smallest x -coordinate, the vertices appear in increasing order of x , which is exactly the sorted order of the input.

Cost of the reduction.

Building the n points costs one multiplication and one assignment each, so $O(n)$ operations. Finding the leftmost point in the output costs one scan, $O(n)$, and reading off the coordinates costs $O(n)$. So apart from the call to the convex hull algorithm, the reduction spends $O(n)$ operations.

Problem 7: Stooge Sort, Slowed Down

Suggested/Proposed by Vardhan Kumar Ray

Stooge sort is a sorting algorithm notable for being remarkably slow: it sorts three overlapping segments, each of about two-thirds the length. The variant below is slower still. It puts the two end entries in order, and then, on a segment of more than three entries, recursively sorts *four* overlapping segments, each of about three-quarters the length: the leading three-quarters and the trailing three-quarters, alternately, twice each.

Algorithm 4 QUADSTOOGES, called as QUADSTOOGES(A, i, n)

```

1: function QUADSTOOGES( $A, i, j$ )
2:    $n \leftarrow j - i + 1$ 
3:   if  $A[i] > A[j]$  then
4:     swap  $A[i]$  and  $A[j]$ 
5:   end if
6:   if  $n = 3$  then                                     ▷ base case: three entries
7:     if  $A[i] > A[i + 1]$  then
8:       swap  $A[i]$  and  $A[i + 1]$ 
9:     end if
10:    if  $A[i + 1] > A[j]$  then
11:      swap  $A[i + 1]$  and  $A[j]$ 
12:    end if
13:  else if  $n \geq 4$  then
14:     $k \leftarrow \lfloor n/4 \rfloor$ 
15:    QUADSTOOGES( $A, i, j - k$ )                           ▷ leading three-quarters
16:    QUADSTOOGES( $A, i + k, j$ )                             ▷ trailing three-quarters
17:    QUADSTOOGES( $A, i, j - k$ )
18:    QUADSTOOGES( $A, i + k, j$ )
19:  end if
20: end function

```

- Write down a recurrence for the running time on an array of n entries and solve it, showing that the running time is $\Theta(n^{\log_{4/3} 4})$.
- Give the best upper bound you can on the total number of swaps performed.
- Show that the fourth recursive call may be deleted without affecting correctness, and say what the running time becomes.

Solution.

Throughout, fix a call on a segment of length $n = j - i + 1$ with $n \geq 4$, put $k = \lfloor n/4 \rfloor$, and split the segment into three consecutive blocks

$$L = A[i \dots i + k - 1], \quad M = A[i + k \dots j - k], \quad R = A[j - k + 1 \dots j],$$

of sizes k , $n - 2k$ and k . The leading three-quarters is $L \cup M$ and the trailing three-quarters is $M \cup R$, both of length $n - k$.

Two facts will be used repeatedly. First, $k \geq 1$, so $n - k < n$ and the recursion makes progress. Second,

$$|M| = n - 2k \geq n - \frac{n}{2} = \frac{n}{2} \geq 2k,$$

so the middle block is at least as large as either end block; in particular $|M| \geq |R|$.

(a) The running time

Outside its recursive calls an invocation does a constant amount of work. Each of the four recursive calls is on a segment of length $n - \lfloor n/4 \rfloor \approx 3n/4$. Ignoring floors,

$$T(n) = 4T\left(\frac{3n}{4}\right) + \Theta(1), \quad T(n) = \Theta(1) \text{ for } n \leq 3.$$

Recursion tree: Each node has four children and the length shrinks by a factor $3/4$ per level, so level t holds 4^t nodes, each of length $(3/4)^t n$ and each costing $\Theta(1)$. The tree bottoms out at depth $d = \log_{4/3} n$.

level t	nodes	length of each	cost of the level
0	1	n	$\Theta(1)$
1	4	$3n/4$	$\Theta(4)$
2	16	$9n/16$	$\Theta(16)$
t	4^t	$(3/4)^t n$	$\Theta(4^t)$

The level costs form a geometric series of ratio $4 > 1$, so the sum is dominated by its last term:

$$T(n) = \Theta\left(\sum_{t=0}^d 4^t\right) = \Theta(4^d) = \Theta(4^{\log_{4/3} n}) = \Theta(n^{\log_{4/3} 4}),$$

using $a^{\log_b c} = c^{\log_b a}$.

Check with the Master theorem (See Quiz 2 Problem 1 for the statement): With $a = 4$, $b = 4/3$ and $f(n) = \Theta(1)$, the critical exponent is $n^{\log_{4/3} 4} \approx n^{4.82}$, and $f(n) = \Theta(1) = O(n^{\log_{4/3} 4 - \varepsilon})$ for any $\varepsilon < 4.82$. Case 1 gives $T(n) = \Theta(n^{\log_{4/3} 4})$, agreeing with the tree.

(b) The number of swaps

See solution of problem 3 (b).

(c) The fourth call is redundant

We show that after the first three calls the whole segment is already sorted, so the fourth call changes nothing.

Let T be the set of the k largest entries of the segment.

After the first call the block $L \cup M$ is sorted. Say t members of T lie in $L \cup M$; being the largest entries of the whole segment, they are in particular the t largest of $L \cup M$, so sorting puts them in the last t positions of $L \cup M$. Since $t \leq k \leq |M|$, those positions lie inside M . Hence every member of T now lies in $M \cup R$.

After the second call the block $M \cup R$ is sorted. It contains all k members of T , which are its k largest entries, so they occupy its last k positions, which are exactly R . So R now holds the k largest entries of the segment, in sorted order, and every entry of $L \cup M$ is at most every entry of R .

After the third call the block $L \cup M$ is sorted again, and it holds the $n - k$ smallest entries. The block R is disjoint from $L \cup M$, so the third call did not touch it. So $L \cup M$ is sorted, R is sorted, and everything in $L \cup M$ is at most everything in R : the segment is sorted.

Consequently the fourth call is applied to an already sorted segment and performs no swaps, though it still spends its comparisons. Deleting it gives

$$T(n) = 3T\left(\frac{3n}{4}\right) + \Theta(1) = \Theta(n^{\log_{4/3} 3}),$$

by the same recursion tree with branching factor 3 instead of 4. Note $\log_{4/3} 4 = \log_{4/3} 3 + 1$, so the redundant call costs exactly one full 6power of n .